Preventify Bot

Conversation & API Architecture

Version:	1.0
Date:	2026-05-23
Status:	Approved for Engineering Build
Project:	Preventify Diabetes Educator AI -- Kerala

Internal engineering reference. Do not share outside the Preventify engineering and clinical team.

Preventify Bot -- Conversation & API Architecture

Version: 1.1

Date: 2026-05-23

Status: Approved for Engineering Build -- Testing Phase Active

Table of Contents

1. Purpose & North Star

What this bot does

The Preventify bot is a **conversational diabetes education assistant**. Its job is to answer genuine health questions from people managing diabetes -- in plain language, at the level of a trained diabetes educator. It is currently being tested via a web-based chat frontend; WhatsApp is the production channel, integrated once the base model is validated.

Why it exists (business goal)

The bot runs a **silent lead identification layer** alongside the clinical education layer. Patients never experience this as a sales flow. The bot identifies patients who need more help than a chatbot can provide, earns their trust through genuine answers, and -- at the right moment -- connects them to the nearest Sugar Care Clinic.

```
Patient gets genuine help
    -> Trust builds over multiple sessions
    -> Bot scores engagement + concern depth silently
    -> When score threshold is met -> consent moment
    -> AI brief generated -> pushed to Sugar Care Clinics CRM
    -> Clinic advisor contacts patient
    -> Patient enrolled at nearest clinic
```

North-star metric

Cost per HbA1c-point reduction -- every architectural decision should be evaluated against this.

Hard clinical constraints (non-negotiable)

The system must **never**:

- Recommend a specific drug dose or titration
- Substitute or stop a patient's medication
- Make a diagnosis
- Interpret lab results without clinical context
- Claim to be a doctor or RMP

These are enforced at three independent points: system prompt, constraint check after reranking, and response post-processor.

---

2. Platform: Testing Frontend -> WhatsApp

2.1 Current Phase: Web-Based Testing Frontend

Status: Active -- base model validation in progress

The bot is currently accessed through a **standard web chat interface** (browser-based). This is the testing environment used to validate all bot behaviour -- clinical accuracy, conversation flow, QDS scoring, RAG retrieval, lead capture logic, and safety escalations -- before any production channel is connected.

Why frontend-first:

The base model must be clinically correct and behaviourally sound before it touches real patients on WhatsApp. A controlled chat interface lets the team run structured test cases, observe full pipeline outputs, and iterate on prompts, retrieval logic, and escalation rules without the constraints of the Meta platform.

Testing frontend requirements:

Requirement	Detail
Input	Free text chat input -- no voice in this phase
Output	Bot response rendered as plain text; button/list choice

Session identity	user_id (UUID, generated per tester session) -- replaces
Persistence	Full Neon DB schema active -- profile, session memory, Q
Logging	All turns logged with timestamps, QDS scores, chunk IDs
Access	Internal team only -- Dakshita, Dr. Rakesh, and any clin

What stays identical between testing and production:

Everything from the risk engine inward -- Phase 1 (Context Engine), Phase 2 (RAG Pipeline), the Neon DB schema, lead scoring, and consent flow are built once and shared. The frontend is only the transport layer.

2.2 Production Phase: WhatsApp Integration (Post Base-Model Sign-Off)

Status: Planned -- not active

Once the base model passes clinical validation (Dr. Rakesh sign-off on QDS classification accuracy, clinical answer quality review, red-flag escalation testing), the channel layer switches to WhatsApp. No changes to the RAG, risk engine, or lead capture code -- only the transport layer is added.

Decision: Meta Cloud API Direct (WhatsApp Business Platform)

The bot's traffic profile is almost entirely inbound-initiated -- patients send messages, the bot responds within the 24-hour service window. Under Meta's pricing (effective July 2025), these are **Service messages -- free and unlimited**. The only paid messages are outbound utility templates (e.g. appointment reminders), billed at ?0.115 per message.

BSP options (Twilio, Wati, Interakt) add cost on top:

- Twilio charges ?0.42 per message sent and received -- on top of Meta's rates. At 10,000 inbound service messages/month, that is ?4,200 in overhead for messages Meta gives free.
- Wati gates webhook access behind higher plan tiers.
- Interakt same issue -- webhook access requires Growth plan.

Meta Cloud API Direct gives full feature access at zero platform fee. Setup takes 4-12 weeks including business verification.

Fallback for fast pilot: Use **Interakt on the Growth plan** if the team needs to go live in under 2 weeks. Migrate to Direct Meta once volume is established.

2.3 Message Types (WhatsApp -- Production Phase)

Outbound (bot -> patient)

Type	When to use	Limits
Text	Clinical answers, explanations, follow-up advi	No limit on length
Interactive -- Buttons	Binary or 3-way choices	Max 3 buttons per message
Interactive -- List	Multiple options (symptoms, meal types, etc.)	Max 10 options per list

Template (Utility)	Outbound first contact, appointment remindersMust be pre-approved by Meta

Inbound (patient -> bot)

Type	Bot must handle
Text	Primary input -- always accept
Button reply	Patient tapped a quick reply button
List reply	Patient selected from a list message
Voice note	Malayalam ASR -- Phase 2 language layer (not in first bu
Image	Foot photo for wound screening -- deferred

2.4 Webhook Requirements (WhatsApp -- Production Phase)

The bot receives all inbound WhatsApp events at a single HTTPS webhook endpoint:

- Must return HTTP 200 OK within 5 seconds of receiving any webhook event
- All processing that takes longer than 5 seconds must be handled asynchronously
- Failed webhooks (5 consecutive failures) cause Meta to disable the endpoint -- retry queues and dead-letter handling are required
- Webhook verification: Meta sends a GET request with hub.challenge on setup

2.5 Compliance Notes (DPDP 2023) -- WhatsApp Phase

- Preventify is the Data Fiduciary -- Meta/WhatsApp is the processor
- Explicit, purpose-limited, auditable consent must be captured before any clinical data is stored
- PHI must not be stored in WhatsApp message logs -- keep in Preventify's own database only
- Meta does not sign a clinical data processing agreement -- PHI must be kept out of the WhatsApp message payload

3. Model Selection

Stack: Google Gemini -- switched from Anthropic Claude. All two-phase architecture and caching strategy remain identical; only the model names and pricing change.

3.1 Phase 1 -- Context Engine: Gemini 2.0 Flash

Model: gemini-2.0-flash-001

What Phase 1 does: Runs on every single message. Classifies intent, assigns QDS score, checks if context is sufficient to search the vector database, generates clarifying questions if not, and extracts profile signals.

Why Gemini 2.0 Flash:

- Fastest Gemini model -- 500-800ms TTFT, critical for chat UX
- Cheapest at $0.10 / $0.40 per million input/output tokens -- 10x cheaper input than Claude Haiku
- Classification, scoring, and JSON extraction are well within Flash's capability
- Context caching at $0.025/M tokens -- 0.25x of standard input price on cache hits
- 1M token context window

Gemini context caching -- minimum token requirement:

Gemini's context cache has a **1,024-token minimum**. The Phase 1 system prompt alone (~600 tokens) falls below this. **Solution:** the cached block must include system prompt + full QDS rubric with annotated examples, expanded to >=1,100 tokens. This is the correct engineering approach -- more detailed examples also improve QDS accuracy, so the expansion serves both purposes.

Phase 1 token budget per turn (estimated):

Component	Tokens	Cached?
System prompt + QDS rubric + annotated exa	~1,100	Yes -- qualifies for Gemini context cache
Patient message	~50	No
Output (JSON: intent, qds, sufficiency, questic	~40	--
Cost per turn (cache hit)	--	~$0.000049
Cost per turn (cache miss / first call)	--	~$0.000131

3.2 Phase 2 -- RAG Response: Gemini 2.5 Pro

Model: gemini-2.5-pro-preview-06-05

What Phase 2 does: Runs only when Phase 1 determines context is sufficient. Receives the top-5 retrieved clinical chunks plus the patient's message and conversation history. Generates the clinical education response with safety constraints enforced.

Why Gemini 2.5 Pro:

- Best reasoning model in the Gemini family -- strong safety-constraint enforcement, catches edge cases where a patient question appears benign but carries clinical risk
- Excellent long-context synthesis -- reads and reconciles multiple clinical guideline chunks (RSSDI vs ADA vs KDIGO) into one coherent patient-friendly answer
- 1M token context window -- full conversation history fits without truncation
- $1.25 / $10.00 per million input/output tokens (<=200K context) -- meaningfully cheaper than Sonnet 4.6 on output ($10 vs $15 per million)
- Context caching at $0.3125/M tokens

Gemini context caching -- Phase 2:

Same 1,024-token minimum applies. Phase 2 system prompt alone (~800 tokens) is just under. **Solution:** cached block = system prompt + clinical scope rules + safety constraint examples + bot persona detail, expanded to ~1,100 tokens. The additional detail (more constraint examples) improves safety enforcement -- dual benefit.

Phase 2 token budget per turn (estimated):

Component	Tokens	Cached?
System prompt + safety rules + clinical scope	~1,100	Yes
Top-5 RAG chunks	~1,200	No (query-specific)
Conversation history (last 5 turns)	~500	No
Short patient memory (profile summary)	~100	No
Patient message	~50	No
Output (clinical response)	~300	--
Cost per turn (cache hit on system prompt)	--	~$0.006
Cost per turn (cache miss)	--	~$0.007

3.3 Context Caching -- Implementation Notes

SDK: Use the **Google AI Python SDK** (google-generativeai) or **Vertex AI SDK** -- both support context caching. Google AI SDK is simpler for initial build; Vertex AI gives better enterprise controls (IAM, audit logs) for production.

Cache TTL: Configurable -- set to **10 minutes** (vs Anthropic's fixed 5-min TTL). Longer TTL means more cache hits for patients who send messages in bursts within a session. Storage cost is $1.00/M tokens/hr (Flash) and $4.50/M tokens/hr (2.5 Pro) -- at ~1,100 tokens per system prompt, the storage cost is negligible ($0.0000011/hr for Flash, $0.0000049/hr for 2.5 Pro).

Cache key strategy:

- Phase 1: one cached object per deployment -- the system prompt + QDS rubric never changes per turn
- Phase 2: one cached object per deployment -- the system prompt + safety rules never changes per turn
- Do NOT attempt to cache RAG chunks (they are query-specific and change every turn)

Cache write: First call per TTL window pays standard input price (1.0x) to write the cache. All subsequent calls within the TTL window pay 0.25x (Flash) or 0.25x (2.5 Pro). With ~200 messages/day and a 10-minute TTL, expect ~80-90% of Phase 1 calls to hit cache within active sessions.

3.4 Cost Estimate

Assumptions: 200 patient messages/day, 50% trigger Phase 2, 80% Phase 1 cache hit rate, 80% Phase 2 cache hit rate.

Phase 1 daily cost (Gemini 2.0 Flash, 200 turns):

```
200 x [0.8 x $0.000049 + 0.2 x $0.000131]
= 200 x [$0.0000392 + $0.0000262]
= 200 x $0.0000654
= $0.013/day
```

Phase 2 daily cost (Gemini 2.5 Pro, 100 turns):

```
100 x [0.8 x $0.006 + 0.2 x $0.007]
= 100 x [$0.0048 + $0.0014]
= 100 x $0.0062
= $0.62/day
```

Configuration	Daily cost	Monthly cost
Flash (Phase 1) + 2.5 Pro (Phase 2) -- with ca	~$0.63	~$19
Flash (Phase 1) + 2.5 Pro (Phase 2) -- no caching	~$0.72	~$22
2.5 Pro for both phases -- no caching	~$1.27	~$38

Decision: Flash for Phase 1, 2.5 Pro for Phase 2, with Gemini context caching enabled.

Output token cost is the dominant cost driver -- Phase 2 output (300 tokens x $10/M) accounts for ~$0.003 of every Phase 2 turn. This cannot be cached and scales directly with message volume. At 1,000 messages/day (future scale), expect ~$3/day on output alone.

4. Conversation Flow Architecture

4.1 Two-Phase Design

The key design principle: **do not search the vector database until the patient's context is understood.**

A doctor does not immediately look up a textbook when a patient mentions a symptom. They first listen, clarify if needed, form a mental model of the patient's situation -- and then decide what clinical guidance is relevant. The bot follows the same pattern.

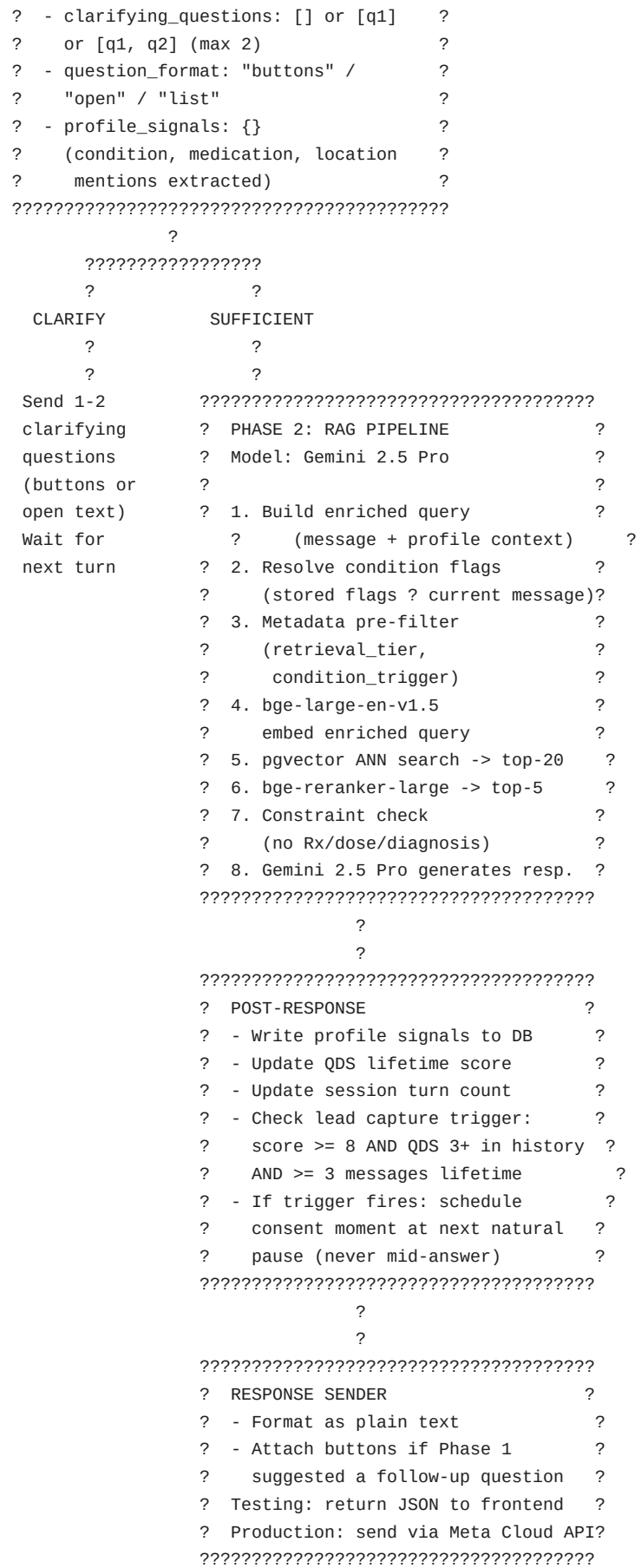
```
Patient's actual words: "mere paon mein dard hai" (feet hurting)

? Naive approach: immediately search "foot pain diabetes"
   -> returns generic neuropathy chunks
   -> misses: is it a wound? burning sensation? new or chronic? any sores?

v Two-phase approach:
   -> Phase 1: classify as QDS 4 (complication concern), flag insufficient context
   -> Ask: "Is it more of a burning/tingling feeling, or sharp pain?
            And have you noticed any wounds or sores on your feet?"
   -> With answer: build enriched query "diabetic foot neuropathy burning tingling,
                   no visible wound, CKD in profile"
   -> KDIGO + RSSDI + ADA all inform the answer
```

4.2 Full Pipeline

```
HTTP Request Received
(Testing: web chat frontend POST /chat)
(Production: Meta Cloud API webhook)
         ?
         ?
??????????????????????????????????????????
?  REQUEST HANDLER                       ?
?  - Parse message type                  ?
?    (text / button_reply / list_reply / ?
?     voice / image)                     ?
?  - Extract: user_id, content,          ?
?    message_id, timestamp               ?
?  - Return HTTP 200 immediately         ?
?  - Push to async processing queue      ?
??????????????????????????????????????????
               ?
               ?
??????????????????????????????????????????
?  SESSION MANAGER                       ?
?  - Load user profile from Neon DB      ?
?    (Layer 1, 2, 3)                     ?
?    (Testing: keyed by user_id/UUID)    ?
?    (Production: keyed by WhatsApp num) ?
?  - Load last 5 turns of current        ?
?    session (in-memory, current session ?
?    only)                               ?
?  - Load short persistent memory        ?
?    (~100 tokens of compressed profile) ?
?  - Check: is this a mid-clarification  ?
?    turn? (bot asked a question last    ?
?    turn -- patient is answering it now) ?
??????????????????????????????????????????
               ?
               ?
??????????????????????????????????????????  <- Always runs, never waits
?  RISK ENGINE (deterministic)           ?
?  - Scan message for red-flag keywords  ?
?  - Assign Tier 0-4                     ?
?  - Tier 4 (emergency): bypass all      ?
?    phases immediately -> send emergency ?
?    safety message + notify RMP         ?
??????????????????????????????????????????
               ? (if not Tier 4)
               ?
??????????????????????????????????????????
?  PHASE 1: CONTEXT ENGINE               ?
?  Model: Gemini 2.0 Flash               ?
?                                        ?
?  Input:                                ?
?  - Current patient message             ?
?  - Short persistent memory (~100 tok)  ?
?  - Last 5 turns (current session)      ?
?  - Conversation state flag             ?
?                                        ?
?  Output (JSON):                        ?
?  - intent: string                      ?
?  - qds_score: 1-5                      ?
?  - context_sufficient: true/false      ?
```


- clarifying_questions: [] or [q1]
or [q1, q2] (max 2)
- question_format: "buttons" /
"open" / "list"
- profile_signals: {}
(condition, medication, location
mentions extracted)
CLARIFY
SUFFICIENT
Send 1-2
clarifying
questions
(buttons or
open text)
Wait for
next turn
PHASE 2: RAG PIPELINE
Model: Gemini 2.5 Pro
1. Build enriched query
(message + profile context)
2. Resolve condition flags
(stored flags ? current message)
3. Metadata pre-filter
(retrieval_tier,
condition_trigger)
4. bge-large-en-v1.5
embed enriched query
5. pgvector ANN search -> top-20
6. bge-reranker-large -> top-5
7. Constraint check
(no Rx/dose/diagnosis)
8. Gemini 2.5 Pro generates resp.
POST-RESPONSE
- Write profile signals to DB
- Update QDS lifetime score
- Update session turn count
- Check lead capture trigger:
score >= 8 AND QDS 3+ in history
AND >= 3 messages lifetime
- If trigger fires: schedule
consent moment at next natural
pause (never mid-answer)
RESPONSE SENDER
- Format as plain text
- Attach buttons if Phase 1
suggested a follow-up question
Testing: return JSON to frontend
Production: send via Meta Cloud API

5. Opening Message

This is the message the bot sends when a patient messages for the first time, or sends a greeting ("Hi", "Hello", "Namaste").

Decided opening message (English -- v1):

```
? Namaste! I'm a diabetes health guide from Preventify.

I can help with questions about managing diabetes --
food, exercise, medications, and day-to-day life.

What's on your mind today?
```

Design principles applied:

- No clinical terminology in the opening
- No form-like questions upfront
- Open-ended -- patient decides where to start
- No quick reply buttons on the opening -- let them speak naturally
- Short -- users do not read long first messages (applies to both testing frontend and WhatsApp)

Requires before go-live: Clinical language review, Malayalam translation.

6. Phase 1 -- Context Engine

6.1 What It Does

Phase 1 is a lightweight Gemini 2.0 Flash call that runs on **every message** before any vector search happens. It answers one question: *"Do I understand this patient's situation well enough to retrieve the right clinical information?"*

If yes -> proceed to Phase 2 (RAG).

If no -> ask at most 2 clarifying questions, wait for the patient's reply.

Phase 1 also extracts any profile signals from the message (condition mentions, medication mentions, location mentions) -- these are written to the user profile regardless of whether Phase 2 runs.

6.2 QDS Scoring

Score	Intent	Patient example
1	General awareness -- no personal stake	"What is HbA1c?"
2	Personal relevance -- applying to their situation	"My HbA1c is 7.2 -- is that okay?"
3	Active management -- making decisions	"Should I take my tablet before or after food?"

4	Complication concern -- worried about complications	"My feet go numb at night"
5	Complex / distressed -- multiple concerns or e	"Doctor wants to put me on insulin and I'm scared"

Important: QDS is assigned by the LLM, not keyword matching. Patients do not use clinical language. "Doctor said my sugar is going up again" is QDS 3, not QDS 1, because it signals active management failure.

6.3 Context Sufficiency Rules

These are the initial rules. They will be tuned during testing.

Situation	Clarify?	Reason
QDS 1 -- pure definition question	No -> search	Context does not change a definition
QDS 2 -- personal relevance, profile is known	No -> search	Profile fills the gap
QDS 2 -- personal relevance, new user	No -> search	Generic answer is acceptable for first contact
QDS 3 -- active management, medication unclear	Yes	Which tablet? Timing context matters
QDS 3 -- active management, medication clea	No -> search	Profile already holds medication info
QDS 4 -- foot symptom	Yes	Neuropathy vs wound vs infection needs one question
QDS 4 -- dizziness	Yes	Low sugar vs BP drop vs dehydration -- different answers
QDS 5 -- distressed, emotional signal	No -> search immediately	Answer first. Never ask questions when patient is distr
Any Tier 3/4 risk flag	No -> escalate	Risk engine bypasses Phase 1 logic

6.4 Clarifying Question Format Guide

When to use buttons (max 3):

- The answer is one of 2-3 known options
- Patient needs to choose, not describe
- Example: "Is the pain more of a burning/tingling feeling, or sharp pain?"
  - Button 1: Burning / Tingling
  - Button 2: Sharp Pain
  - Button 3: Both

When to use a list (max 10):

- Multiple options with distinct meanings
- Example: "Which meal does this happen after?"
  - Morning tea/breakfast
  - Lunch
  - Dinner
  - Between meals
  - Not related to meals

When to use open text (no buttons):

- Patient needs to describe, not choose
- Emotional or distressed messages -- never give options here
- Duration questions ("How long has this been happening?")

- Any question where the right answer might not be in our list

Hard rule: Maximum 2 clarifying questions per turn. Never 3. Never send two separate messages -- combine both questions into one message if 2 are needed.

6.5 Profile Signal Extraction

Phase 1 extracts profile signals from every message, regardless of whether clarification is needed. These are written to the patient's profile in the database.

Signals extracted:

Signal type	Patient says	What is stored
Diabetes type	"doctor told me sugar is high"	diabetes_suspected: true
Diabetes type	"I have Type 2" / "sugar patient"	diabetes_type: T2DM
Medication	"I take white tablet in morning"	medications_mentioned: [oral_antidiabetic_unspecified]
Medication	"I take metformin"	medications_mentioned: [metformin]
Medication	"I take injection"	insulin_user: true
Complication	"feet go numb" / "burning in legs"	complication_flags: [neuropathy_suspected]
Complication	"kidney problem" / "creatinine is high"	condition_flags: [ckd]
Complication	"heart problem" / "BP is high"	condition_flags: [cardio]
Fasting context	"Ramadan" / "roza"	condition_flags: [ramadan]
Location	"I'm in Thrissur" / "nearest clinic is far"	location_hint: Thrissur
Family context	"my father has diabetes"	session_context: family_member_inquiry

Signals are extracted by the LLM, not keyword matching. The LLM understands "I take one white tablet in morning" as a medication signal even though no drug name is mentioned.

7. Phase 2 -- RAG Pipeline

7.1 Query Construction: New vs Returning User

The enriched query is what gets embedded and sent to pgvector. It is **never shown to the patient.**

New user (no stored profile):

```
retrieval_query = current_message
# "Can I eat rice?"
```

No enrichment -- the patient's profile is empty. Generic retrieval is correct for first contact.

Returning user (profile exists):

```
def build_enriched_query(message: str, profile: UserProfile) -> str:
    context_parts = []
    if profile.diabetes_type:
        context_parts.append(profile.diabetes_type)
    if profile.condition_flags:
        context_parts.append(", ".join(profile.condition_flags))
    if profile.medications_mentioned:
        context_parts.append(f"on {profile.medications_mentioned[0]}")
    if context_parts:
        return f"{message} [Patient context: {'; '.join(context_parts)}]"
    return message

# Result: "Can I eat rice? [Patient context: T2DM; CKD; on metformin]"
```

The [Patient context: ...] suffix helps the embedding model find more relevant chunks. For a patient with CKD, the query now surfaces KDIGO protein/carbohydrate guidance that a plain "Can I eat rice?" query might not reach.

7.2 Condition Flag Resolution

Condition flags determine which Tier-2 sources open up for this query. They are resolved **before** the pgvector search.

```
def resolve_condition_flags(message: str, profile: UserProfile | None) -> set[str]:
    flags = set()

    # Always check current message (new and returning users)
    if contains_ckd_signal(message):          flags.add("ckd")
    if contains_cardio_signal(message):       flags.add("cardio")
    if contains_ramadan_signal(message):      flags.add("ramadan")
    if contains_hypertension_signal(message): flags.add("hypertension")

    # Returning user: also include stored profile flags
    if profile and profile.condition_flags:
        flags |= set(profile.condition_flags)

    return flags
```

Stored flags are permanent. A patient who mentioned kidney problems in session 3 continues to have KDIGO included in retrieval in session 10, even if the current message says nothing about kidneys. Medical conditions do not disappear.

7.3 Metadata Pre-filter

The pre-filter is the WHERE clause on pgvector -- it narrows the search pool before ANN search runs.

```
def build_retrieval_filter(flags: set[str]) -> dict:
    if not flags:
        # No condition flags -- Tier 1 only
        return {"retrieval_tier": "core"}
    else:
        # Condition flags fired -- include Tier 2 sources for those conditions
        return {
            "retrieval_tier": ["core", "triggered"],
            "condition_trigger": [None] + list(flags)
        }
```

Tier 1 (core -- always searched):

- RSSDI 2022 -- first-choice for all standard T2DM queries
- ICMR STW 2024 -- current GoI clinical decision flow
- ADA 2026 -- fallback when India sources are silent
- ICMR-NIN -- nutrition and food queries
- Anoop Misra -- South Asian dietary guidelines

Tier 2 (triggered -- searched only when flag fires):

- KDIGO 2022 -- ckd flag
- IDF-DAR -- ramadan flag
- ESC 2023 -- cardio flag
- WHO HEARTS -- hypertension flag

7.4 Retrieval

```
Enriched query
    -> bge-large-en-v1.5 (1024-dim embedding)
    -> pgvector ANN search with metadata pre-filter
    -> top-20 candidates
    -> bge-reranker-large (cross-encoder, scores each pair)
    -> top-5 chunks
    -> Constraint check (no Rx/dose/diagnosis language)
    -> Gemini 2.5 Pro response generation
```

Both stages are required. The embedder is fast but approximate; the reranker is slower but precise. Skipping either degrades clinical quality.

7.5 Response Generation

Gemini 2.5 Pro receives:

- System prompt (context-cached): safety rules, bot persona, clinical scope, escalation triggers
- Short patient memory (~100 tokens): compressed profile summary
- Last 5 turns of current session
- Top-5 retrieved chunks
- Patient's current message

Gemini 2.5 Pro does **not** receive the enriched query -- only the original patient message. The enrichment was for retrieval only.

Constraint enforcement inside Gemini 2.5 Pro's system prompt:

- Never recommend a specific drug dose
- Never tell a patient to stop or change their medication
- Never make a diagnosis
- If a question requires clinical judgment beyond DSMES scope -> escalate to Tier 2 or higher, recommend clinic visit

8. User Profile Schema

8.1 Three Layers (Stored in Neon PostgreSQL)

Layer 1 -- Identity

Field	Type	Notes
user_id	TEXT (PK)	Testing phase: UUID generated per session. Production (
name	TEXT	Collected at consent or mentioned naturally
age	INT	Collected via one natural question early in first sessi
first_contact_date	TIMESTAMPTZ	Auto-set on first message
location_hint	TEXT	City/area inferred from conversation or given at consen

Layer 2 -- Clinical Profile

Field	Type	Notes
diabetes_type	TEXT	T1DM / T2DM / GDM / Prediabetes / Undiagnosed / null
condition_flags	TEXT[]	ckd, cardio, ramadan, hypertension -- permanent once set
medications_mentioned	TEXT[]	Drug names or categories mentioned by patient
insulin_user	BOOLEAN	True if patient mentions injections
complications_mentioned	TEXT[]	neuropathy_suspected, retinopathy_suspected, etc.
highest_qds_ever	INT	Peak QDS asked across all sessions
escalation_history	JSONB	Log of Tier 3/4 events -- date, trigger, outcome

Layer 3 -- Engagement

Field	Type	Notes
lifetime_score	FLOAT	QDS-based score with volume decay and recency weighting
total_sessions	INT	Count of distinct sessions
total_messages	INT	Count of all messages lifetime
last_session_date	DATE	For recency weighting
consent_status	TEXT	not_yet / given / declined
consent_timestamp	TIMESTAMPTZ	When consent was given
consent_declined_at	TIMESTAMPTZ	When decline was recorded

lead_status	TEXT	new_lead / contacted / qualified / converted / closed

8.2 Session Memory (Last 5 Turns)

- Stored in-memory for the duration of the current session only
- Not persisted to the database
- Format: list of {role: "patient"/"bot", content: "..."} objects
- Maximum 5 turns (10 messages total -- 5 patient + 5 bot)
- Older turns within the session are dropped as the window moves
- When the session ends, the full session history is not stored -- only the compressed short memory (below) is written to the database

8.3 Short Persistent Memory (~100 tokens)

This is what gets passed to the LLM on every Phase 2 call to give it patient context without sending the full profile. Compressed at the end of each session by Haiku (cheap call).

Format:

```
Patient: [name if known], [age if known]
Condition: [diabetes_type if known, or "sugar problem reported"]
Flags: [condition_flags if any]
Medications: [medications_mentioned if any]
Peak concern: [highest QDS topic]
Sessions: [total_sessions]
```

Example:

```
Patient: Rajan, 58
Condition: T2DM (inferred session 2)
Flags: CKD (session 3), Neuropathy suspected (session 5)
Medications: Metformin (mentioned), some BP tablet (name unknown)
Peak concern: Foot numbness (QDS 4, session 5)
Sessions: 6
```

This is ~80 tokens. Gives Gemini 2.5 Pro meaningful context without sending the full database record.

9. Condition Flags

9.1 Flags Are Permanent Once Fired

Medical conditions do not disappear. Once a condition flag is set in a patient's profile, it stays set permanently and continues to trigger the relevant Tier-2 sources in every future session.

Flag	Triggered by	Tier-2 source it opens
ckd	Kidney / creatinine / eGFR / dialysis / protein i	KDIGO 2022
cardio	Heart disease / chest pain / angina / heart failure	ESC 2023
ramadan	Ramadan / roza / fasting (religious context)	IDF-DAR 2021
hypertension	High BP / blood pressure problem / BP tablet	WHO HEARTS

9.2 Conflict Resolution

Scenario: Patient mentioned kidney problem in session 2. In session 4 they say "no kidney issues actually."

Rule: Do not clear the flag. Do not ignore the patient's statement either.

Approach: The next time a kidney-relevant query comes up, the bot asks a natural re-confirmation question before generating a CKD-specific answer.

```
Patient asks: "Can I eat more protein?"

[Internal: CKD flag is set, but patient previously said "no kidney issues"]

Bot asks: "I want to make sure I give you the right information about
protein -- has your doctor mentioned anything about your kidneys or
creatinine levels recently?"
```

If the patient confirms no kidney problem -> update profile to note the clarification. If they re-confirm kidney involvement -> flag stays active.

This mirrors what a good doctor does: does not dismiss prior history, creates space for the patient to correct it.

10. Demographic Collection Strategy

10.1 Principle: Silent Inference First

The bot infers everything it can from natural conversation. It does not ask clinical questions directly. Patients do not know clinical terminology and should never be made to feel like they are filling in a form.

What patients know vs what we need:

What we need	What patient actually says
Diabetes type	"Doctor said sugar is high" / "Sugar patient"
HbA1c level	"My sugar test came back bad"
CKD	"Some kidney problem" / "Doctor told me to watch protei
Neuropathy	"Feet go numb at night" / "Burning in legs"
On Metformin	"I take one white tablet in the morning"
On Insulin	"I take injection"

The LLM in Phase 1 extracts clinical signals from patient language -- it does not require the patient to use clinical terms.

10.2 Age: One Natural Question

Age significantly changes clinical advice (elderly protocols differ from adults -- ADA Section 12). The bot asks it once, early in the first session, in plain language.

When to ask: After the patient's first substantive message (not in the opening). Only if age has not been collected yet.

How to ask:

```
"Just so I can give you advice that fits your situation --
roughly how old are you? (You don't need to be exact.)"
```

No buttons for this -- open text. Patients may say "58", "around 60", "retired age".

Never ask age again after it is collected, even if the patient gives an approximate answer.

10.3 Location: At Consent Moment

Location is the most important field for the business goal (routing patients to nearby Sugar Care Clinics). But asking for it upfront feels intrusive.

When to ask: At the consent moment -- when the patient has already agreed to be contacted. This is the most natural time because the patient has just said yes to connecting with the clinic.

How to ask (part of the consent flow):

```
"Which area are you based in?
We'll suggest the most convenient clinic for you."
```

Buttons with district names (if Kerala coverage is specific) or open text.

Patients also often mention location naturally in conversation ("nearest hospital is far from Thrissur"). Phase 1 extracts this as a location signal whenever it appears.

11. Risk Engine

The risk engine runs **in parallel** with Phase 1 on every message. It is deterministic -- no LLM involved. It does not wait for Phase 1 to complete.

Tier	Description	Action
0	No concern	Continue to Phase 1 -> Phase 2 normally
1	Low concern	Include gentle nudge toward next clinic visit in respon
2	Moderate concern	Recommend clinic visit within 1-2 weeks
3	High concern	Recommend clinic within 24-48 hours
4	Emergency	Bypass all phases. Send immediate patient safety instru

Tier 4 red flags (examples -- not exhaustive):

- Blood glucose < 70 mg/dL or > 300 mg/dL
- Chest pain
- Loss of consciousness
- Signs of DKA (nausea, fruity breath, confusion)
- Suicidal ideation or self-harm language
- Foot wound with signs of infection

Rule: Clinical escalation always overrides lead capture. A Tier 3/4 event is never routed through the lead capture flow.

12. Lead Capture Layer

This runs silently. Patients never experience it as a sales flow.

12.1 Lifetime Score Calculation

Score accumulates across all sessions with volume decay on low-depth questions:

QDS	Volume	Multiplier
1-2	Questions 1-4	1.0x
1-2	Questions 5-7	0.5x
1-2	Questions 8+	0.25x
3, 4, 5	Any	Always 1.0x

Recency weighting: current session = 1.0x, last 30 days = 0.8x, older = 0.5x.

12.2 Capture Trigger

Both conditions must be true simultaneously:

- Condition A: Lifetime score >= 8 (with decay and recency applied)
- Condition B: At least one QDS 3+ question in lifetime history
- Floor: Minimum 3 total messages before trigger can fire

12.3 Consent Moment Rules

- Never interrupt mid-answer
- Wait for a natural pause after delivering a complete answer
- Consent message must reference a specific topic from the conversation -- never templated
- Collect name (if unknown) and age (if unknown) at this point
- Ask location here (see Section 10.3)
- DPDP 2023 compliant: patient told explicitly what data is used for and by whom
- Consent timestamp stored

12.4 AI Brief (Generated at Consent)

Pushed to Sugar Care Clinics CRM via webhook immediately on consent:

Field	Source
Name	Layer 1 profile
Contact identifier	Layer 1 profile (user_id in testing; whatsapp_number in
Age	Layer 1 profile
Location	Layer 1 profile (location_hint)
Condition type	Layer 2 profile (diabetes_type)
Concern summary	LLM-generated, 2-3 lines, from full conversation histor
Engagement score	Layer 3 (lifetime_score at capture time)
Peak concern	Layer 2 (highest QDS topic ever raised)
Capture timestamp	IST

12.5 Re-prompt on Declined Consent

- Patient declined: no re-prompt in same session
- Returning declined patient: may be re-prompted once when lifetime score exceeds 12

13. Context Caching Strategy (Gemini)

Gemini context caching reduces cached-input cost to **0.25x of standard input price** on cache hits. TTL is configurable; set to **10 minutes** for this system.

Minimum requirement: Gemini context cache requires >=1,024 tokens in the cached block. Both system prompts are deliberately expanded to ~1,100 tokens to qualify (see Section 3.1 and 3.2).

What to cache	Model	Expected hit rate	Cost saving per hit
Phase 1 system prompt + QDS ru	Flash	Very high -- same every turn, all p	~$0.000083 saved per turn vs uncached
Phase 2 system prompt + safety rules + clinical scope (	2.5 Pro	Very high -- same every turn, all patients	~$0.000963 saved per turn vs uncached
Top-5 RAG chunks	--	Not cached -- query-specific, chan	n/a

Chunk-level query caching (application layer): The most common queries (rice portions, HbA1c explanation, chaaya, exercise) will produce the same pgvector results across patients. Use a Neon PostgreSQL cache table: query_hash -> chunk_ids -> expiry_timestamp. On cache hit, skip the pgvector ANN search entirely -- serve stored chunk IDs directly to the reranker. This is independent of Gemini context caching and saves compute, not just token cost.

What not to put in Gemini context cache:

- Patient message (changes every turn)
- Session history (unique per patient per session)
- Short persistent memory (changes as profile updates)
- Enriched query string (changes based on profile)
- RAG chunks (query-specific)

Session handling: With a 10-minute TTL, consecutive messages within an active patient session will almost always hit the cache. A patient who pauses for >10 minutes pays cache write cost on the next message (one-time, same as standard input). This is acceptable -- the savings on all other turns outweigh the occasional miss.

14. Open Decisions

#	Decision	Status	Phase
D1	Malayalam translation layer -- whi	Open -- post base model build	Post-testing
D2	ASR for voice notes -- Whisper fine-tuned vs Google STT	Open -- post base model build	Post-testing
D3	CRM selection -- Zoho CRM Free	Open -- Operations team	Pre-WhatsApp
D4	Webhook retry queue -- Redis Queue vs PostgreSQL-backed	Open -- Engineering	WhatsApp phase only
D5	Green tick (verified business) -- G	Open -- Operations + Legal	WhatsApp phase only
D6	Opening message -- final Malayalam version	Open -- Clinical + Language review	Pre-WhatsApp
D7	QDS classification validation -- Dr.	Open -- Clinical	Current testing phase
D8	Lead capture consent message language -- DPDP-compliant,	Open -- Clinical + Legal	Pre-WhatsApp
D9	Testing frontend tech stack -- Rea	Open -- Engineering	Immediate

15. Build Order

Each step is independent and can be picked up in a new session by stating the step number.

```
Step 1  ??? Testing frontend + API layer          <- CURRENT PHASE
            - Simple web chat interface (browser)
            - POST /chat endpoint: receives {user_id, message}
            - Returns {response, buttons[], qds_score, risk_tier}
              (debug fields visible to testers, hidden in production)
            - user_id is a UUID -- generated per session
            - No webhook, no Meta API in this step
            - [WhatsApp webhook handler replaces this step in production:
               parses Meta events, returns HTTP 200 within 5s,
               handles hub.challenge verification]

Step 2  ??? Neon database schema + session manager
            - users table (Layer 1, 2, 3 fields)
            - session_turns table (last 5 turns, current session only)
            - chunk_cache table (query hash -> chunk_ids -> expiry)
            - Session manager: load profile + session history + short memory

Step 3  ??? Risk engine
            - Hard-coded deterministic rule engine
            - Red-flag keyword library
            - Tier 0-4 assignment
            - Tier 4 bypass: immediate safety message + RMP notification stub

Step 4  ??? Phase 1: Context Engine
            - Gemini 2.0 Flash call with context-cached system prompt
            - Output: intent, qds_score, context_sufficient,
              clarifying_questions, question_format, profile_signals
            - Profile signal writer (updates DB)

Step 5  ??? Phase 2: RAG Pipeline
            - Query enrichment (new vs returning user)
            - Condition flag resolution
            - Metadata pre-filter
            - pgvector ANN search (bge-large-en-v1.5)
            - Reranker (bge-reranker-large, top-20 -> top-5)
            - Constraint check
            - Gemini 2.5 Pro response generation

Step 6  ??? Response formatter + sender
            - Plain text responses
            - Button message builder (max 3 buttons)
            - List message builder (max 10 options)
            - Testing: return JSON response to frontend
            - Production: send via Meta Cloud API

Step 7  ??? Post-response: profile updater + lead scoring
            - QDS lifetime score calculator (with decay)
            - Capture trigger check
            - Consent moment scheduler

Step 8  ??? Short memory compressor
            - End-of-session Gemini 2.0 Flash call (cheap)
            - Compresses session into ~100-token persistent memory
            - Writes to users table

Step 9  ??? Lead capture + AI brief generator
            - Consent message flow
            - AI brief prompt (Gemini 2.5 Pro, from full conversation history)
```

```
              - Webhook push to CRM

  Step 10 ??? Malayalam translation layer (later phase)
              - Input: Malayalam -> English (before Phase 1)
              - Output: English -> Malayalam (after Phase 2)
              - Slots in around existing English pipeline; RAG code unchanged
```

16. Testing Phase -- What Gets Validated Before WhatsApp

Before the WhatsApp channel is connected, the following must be confirmed via the testing frontend:

Validation	Who	Gate
QDS classification accuracy on 50 real patien	Dr. Rakesh K R	Must pass before WhatsApp
Clinical answer quality -- sample of 20 queries across	Dr. Rakesh K R	Must pass before WhatsApp
Red-flag escalation -- all Tier 4 triggers fire co	Engineering + Clinical	Must pass before WhatsApp
Lead capture trigger -- score threshold fires at correct	Engineering	Must pass before WhatsApp
Consent flow -- DPDP-compliant, natural lang	Clinical + Legal	Must pass before WhatsApp
Opening message -- clinical language review	Clinical	Must pass before WhatsApp
RAG retrieval -- Tier 2 sources trigger only on	Engineering	Must pass before WhatsApp

The testing frontend is the only place where debug output (QDS scores, chunk IDs, risk tier) is visible. Production sends clean patient-facing responses only.

End of document. Version 1.1 -- 2026-05-23.